

Project *infrastructure.*

Hub, vault, GitHub orgs, scaffold rules, migration methodology, and the gotchas that bit us — all in one place.

Contents

01 Infrastructure overview

REFERENCE

02 Secrets vault access

REFERENCE

03 GitHub accounts & orgs

REFERENCE

04 New project auto-wiring rule

RULE

05 App migration methodology

RULE

06 Tailwind 4 cascade-layer gotchas

RULE

07 Music House — migration context

PROJECT

08 Music House — trigger secret gap

PROJECT

Infrastructure overview

REFERENCE reference_infrastructure_overview.md

This is the master map. Everything Daniel ships sits on top of a shared platform layer — apps are thin clients of it. Do not duplicate platform pieces inside an app repo.

GitHub orgs (only one is real)

| Org/User | Status | Purpose | | --- | --- | --- | | `daniels-project-space` | **Active org** — default home for everything | All apps + the platform layer + the hub itself | | `danielmabro-new` | User account, no personal repos | gh CLI auth identity on test-vps | | `Daniel-claude-code-new-project-hub` | Empty legacy org | Ignore | | `remoteworkhq` | **Does not exist on GitHub** | Stale name. The local clone of `remote-work-hub` on the VPS has a `git remote` URL pointing to `https://github.com/remoteworkhq/remote-work-hub.git` — works (probably via GitHub auto-redirect) but `gh repo list remoteworkhq` fails. Real owner is `daniels-project-space/remote-work-hub`. If pushing causes auth issues, fix with `git remote set-url origin git@github.com:daniels-project-space/remote-work-hub.git`.

When user says "my org" or refers to project paths without qualifier, they mean `daniels-project-space`.

Repos in `daniels-project-space` (live)

| Repo | Role | VPS path | | --- | --- | --- | | `platform-storage` | R2 SDK package | `@daniels-project-space/platform-storage` | `/home/ubuntu/dev-staging/platform-storage` | | `platform-agents` | Mastra agent runtime, Vercel-deployed | `/home/ubuntu/dev-staging/platform-agents` | | `platform-jobs` | Trigger.dev v3 task layer | `/home/ubuntu/dev-staging/platform-jobs` | | `project-hub` | Umbrella dashboard + **secrets vault** (Convex) | `/home/ubuntu/project-hub-app` | | `remote-work-hub` | Cloud Claude Code launcher (Next.js + Convex + 21st) | `/home/ubuntu/remote-work/remote-work-hub` | | `sandbox-test` | Throwaway POC repo | n/a |

Convex deployments (each app gets its own — never share)

| Deployment URL | App | | --- | --- | | `https://fantastic-roadrunner-485.convex.cloud` | `project-hub` (widgets, projects, **secrets table**) | | `https://groovy-cardinal-733.convex.cloud` | `remote-work-hub` (sandbox sessions, project logs) | | (TBD per new app) | every new app gets a fresh deployment |

Platform-* repos = GitHub TEMPLATES, NOT runtime dependencies

Architecture decision 2026-05-02: Each app is fully self-contained. The `platform-*` repos exist as **GitHub template repositories** — used to bootstrap a per-app starting point with `gh repo create <new>/<app> --template daniels-project-space/platform-X`, then the new app evolves independently. Apps NEVER import from the platform packages at runtime. Mixing all apps' code into shared repos creates coupled deploys and clutter.

Verified 2026-05-02: no existing app imports `@daniels-project-space/platform-*`. The platform repos contain only generic skeletons (R2 wrapper, echo agent, hello task) — no app-specific code lives in them.

`daniels-project-space/platform-storage` (template, isTemplate=true)

Thin S3 wrapper around `@aws-sdk/client-s3`. Each new app vendors its own `src/lib/storage.ts` (copy-paste pattern from this repo) instead of installing the package — keeps the app self-contained. R2 creds come from project-hub vault `cloudflare` service (5 keys: `R2_ACCOUNT_ID`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `R2_ENDPOINT`, `R2_API_TOKEN`). Each app gets its own bucket named `<slug>`.

`daniels-project-space/platform-agents` (template, isTemplate=true)

Mastra v1.31 reference scaffold. Each new app installs `@mastra/core` directly and defines its own agents in `<app>/src/mastra/`, embedded as Next.js API routes (same Vercel deploy serves frontend + agents). Apps do NOT push to platform-agents. Note: this repo currently has a Vercel deployment + `.vercel/` linked from when it was being prototyped as a shared runtime. Whether to keep that deploy live is TBD (decide before any new app would have referenced it — none do currently).

`daniels-project-space/platform-jobs` (template, isTemplate=true)

Trigger.dev v3 reference scaffold. Each new app creates its OWN Trigger.dev project (e.g. `music-house-jobs`, `lofi-jobs`) and puts task definitions in `<app>/src/trigger/`. Per-app project refs and secret keys go in vault under service `trigger` with `scopes: ["<slug>"]`. Apps do NOT push to platform-jobs. Note: a Trigger.dev project `dps-jobs` (ref `proj_oqwizuikmyjdfuzbetda`) was created during prototyping. No tasks are deployed against it from any app. Decide whether to keep it as a sandbox or delete.

Why this changed

The original idea was a single shared platform layer for storage/agents/jobs. In practice it produced: (1) coupled deploys — one app's Mastra change forces redeploy for everyone, (2) cluttered repos as agents/tasks pile up, (3) awkward secret scoping. Per-app isolation via templates fixes all three.

Remote Work Hub flow (cloud Claude Code)

`/home/ubuntu/remote-work/remote-work-hub/` — Next.js 16 + Convex + 21st.dev SDK

- Hub homepage lists `PROJECTS` from `src/lib/projects.ts` (public-safe metadata).
- Click a project → `POST /api/sessions/spawn` → `agentClient().sandboxes.create(...)` with:
- `setupCommands` from `buildSetup()` — git-clones the project repo into `/home/user/workspace/project` using a GitHub PAT injected as `x-access-token`
- `vaultIds` (when wired) — 21st vault IDs to attach project secrets
- Sandbox runs Claude Code agent against the cloned repo; user can edit from phone via the hub UI.
- Convex `sessions` table tracks sandboxId + threadId + status; UI polls every 1s for fresh-response indicator.
- 21st env: `API_KEY_21ST` at `/home/ubuntu/.21st-env` (single key, also present in vault under `21st` service)

Secrets vault — project-hub Convex `secrets` table

URL: `https://fantastic-roadrunner-485.convex.cloud` Schema: `secrets {service, keyName, value, description?, scopes[], aliases[], sourceFiles[]}`, indexed `by_service` and `by_service_and_key` Functions: `secrets:listByService`, `secrets:getOne`, `secrets:summary`, `secrets:bulkInsert`, `secrets:truncate`

Currently anonymous-queryable (no auth gate yet). Pull a service's keys from any shell with:

```
curl -s -X POST 'https://fantastic-roadrunner-485.convex.cloud/api/query' \
-H 'Content-Type: application/json' \
-d '{"path":"secrets:listByService","args":{"service":"cloudflare"},"format":"json"}'
```

91 secrets across 50 services. Confirmed services with keys (2026-05-02): `anthropic`, `openai`, `openrouter`, `gemini`, `groq`, `deepinfra`, `cerebras`, `fal`, `hf`, `replicate`, `elevenlabs`, `pexels`, `supabase`, `postgres`, `db`, `upstash`, `listmonk`, `expo` (8), `next` (3), `nextauth`, `cloudflare` (5), `vercel`, `mastra` (2), `trigger` (5), `21st`, `telegram` (3), `email`, `chatwoot`, `anythingllm`, `binance` (2), `coinbase` (2), `aws` (2), `google` (2), `stripe` (2), `claw`, `fsq`, `hygglo`, `internal`, `secret`, `service` (2), `serpapi`, `truelayer` (3), `yapily` (2), `token`, `auth` (2), `bot`, `anon` (2), `api` (2), `access`, `health`

Not yet in vault (must add when scaffolding apps that need them): `suno`, `mureka`, `kits` (KitsAI). Their values are still in the legacy `/home/ubuntu/passive-income/ai-music-empire/config.json`.

Standard scaffold for every NEW app project

Run all 9 steps at creation. Skipping any breaks mobile workflow, secret hygiene, or platform reuse.

- 1. GitHub repo:** `gh repo create daniels-project-space/<slug> --public` (top-level, NEVER a sub-repo). Default = **PUBLIC** (set 2026-05-02 — Vercel Hobby plan returns `repo_owned_by_org` 409 on private org repos, so Daniel reverted to public until/unless Vercel Pro is purchased). Secrets MUST stay in the vault and out of source either way.
- 2. Local path:** `/home/ubuntu/<slug>/` on test-vps (NOT under `passive-income/`, NOT under `dev-staging/` unless platform-layer)
- 3. Next.js 16 + Convex + Tailwind 4** scaffold (matches `remote-work-hub` and `project-hub-app` stack). Run `npx convex dev` once → fresh deployment URL → record in `.env.local` as `NEXT_PUBLIC_CONVEX_URL`.
- 4. Vercel project (automatic — run from VPS):** pull `VERCEL_TOKEN` from project-hub vault `vercel` service, then `vercel link --token=$VERCEL_TOKEN --yes --project=<slug>` and `vercel --prod --token=$VERCEL_TOKEN`. Auto-deploys on `main` thereafter via GitHub integration. **Never wait for desktop interaction** — this is fully scriptable.
- 5. R2 in-repo:** vendor a thin S3 wrapper into `<app>/src/lib/storage.ts` using `@aws-sdk/client-s3` + `@aws-sdk/s3-request-presigner`. Each app gets its own bucket `<slug>`. Do NOT depend on `@daniels-project-space/platform-storage` — copy the pattern, don't import the package.
- 6. Trigger.dev in-repo:** create OWN Trigger.dev project for the app (e.g. `<slug>-jobs`), put tasks in `<app>/src/trigger/`. Per-app project ref + secret key in vault under `trigger` with `scopes: ["`

`<slug>"]` .

7. **Mastra in-repo (if needed)**: install `@mastra/core` , define agents in `<app>/src/mastra/` , expose as Next.js API routes. Same Vercel deploy serves frontend + agents.
8. **Secrets**: add app-specific keys (Suno, Mureka, etc.) to project-hub Convex via `secrets:bulkInsert` . App pulls them at runtime via `secrets:listByService` .
9. **Hub registration (automatic — run from VPS)**: clone or update `/home/ubuntu/remote-work/remote-work-hub/` , edit `src/lib/projects.ts` to append the new slug to BOTH `PROJECTS` (with `services:string[]` listing vault services the sandbox needs) and `PROJECT_REPOS` maps. The hub's `sessions.ts:buildSetup` will pull those services from the project-hub vault into the sandbox's `.env.local` automatically on spawn. Commit + push from VPS via the existing `gh` CLI auth. **Never wait for desktop interaction**. Repo path always `daniels-project-space/<slug>` .
1. **Mark the app live in `project-hub/src/lib/apps.ts`** (mandatory once Vercel URL exists). Set `status: "live"` , add `vercelUrl` and `githubUrl` . If migrating from a legacy slug, update `slug / name / short / description` and rename the matching key in `src/lib/app-icons.tsx` . This drives the dashboard apps dock's emerald pulse + clickable tile (`app-tile.tsx` reads `vercelUrl` and `STATUS_DOT[status]`). Commit + push to `daniels-project-space/project-hub` .

Verify the hub redeployed. After push, the hub Vercel project must build the new commit — otherwise the new project tile won't appear. If `link.sourceless` on the hub Vercel project is `True` , push events are ignored. Fix:

```
# Disconnect + reconnect Git link to clear sourceless flag
urlLib.request.Request(f'https://api.vercel.com/v4/projects/{HUB_ID}/link?teamId={TEAM}', meth
urlLib.request.Request(f'https://api.vercel.com/v4/projects/{HUB_ID}/link?teamId={TEAM}', meth
    data=json.dumps({'type':'github','repo':'daniels-project-space/remote-work-hub'})).encode()
# Then trigger a one-time manual deploy of the current main HEAD via /v13/deployments
```

After this, `sourceless=None` and pushes auto-deploy. Hub Vercel project ID:

`prj_DG6gJEjYvAsVGm4RAEf5SRtHnbIB` . Production: `https://remote-work-hub-sepia.vercel.app/` .

After step 9 the project is auto-deployable from a `git push origin main` (Vercel CD), accessible from phone via the hub sandbox, and has secrets loaded centrally.

Things that are NOT the default — only invoke when needed

- **VPS systemd services / pm2 / cron** — only if the app has truly always-on local needs. Default = Trigger.dev. Do not stand up a custom worker by default.
- **VPS nginx routing** — only if the app must be served on the rental-manager domain (legacy reasons). Default = Vercel domain. The rental-manager nginx block is for legacy projects; new apps deploy via Vercel.
- **Supabase, Postgres, AWS** — secrets exist in vault for legacy reasons. Default storage is Convex (data) + R2 (media).

Legacy structures still on VPS — reference, don't extend

- `/home/ubuntu/passive-income/<app>/` — old monolithic Node + Python apps (rental-manager, autostudio, ai-music-empire, lofi-generator, ai-instagram, etc.). Most are pre-platform-layer. Migrate them to the new stack rather than extending in place.
- `/home/ubuntu/app-factory/` — autonomous app creator (different concern, has own pipeline)
- `/etc/nginx/sites-enabled/rental-manager` — single big config holding many `location` blocks for legacy routes. Don't add new entries here; deploy new apps to Vercel.

Secrets vault access

REFERENCE `reference_secrets_vault.md`

The "vault" Daniel refers to is the `secrets` table in the **project-hub** Convex deployment. It superseded an older Supabase key-vault project. Every app's API keys, deploy tokens, and infra creds belong here — never hardcoded in repos, never stuffed in per-app `.env` files for production.

Endpoint

<https://fantastic-roadrunner-485.convex.cloud/api/query> (anonymous-queryable in current state — auth gate to be added when project-hub gets user accounts)

Schema

```
secrets {
  service: string,           // "cloudflare", "anthropic", "suno", ...
  keyName: string,          // e.g. "R2_ACCESS_KEY_ID" — what apps actually call os.environ this as
  value: string,            // the literal secret
  description?: string,
  scopes: string[],         // free-form tags
  aliases: string[],        // alternative env-var names the same secret is known as
  sourceFiles: string[]     // optional provenance — files where this leaked from
}
// indexes: by_service, by_service_and_key
```

Read

List one service:

```
curl -s -X POST 'https://fantastic-roadrunner-485.convex.cloud/api/query' \
-H 'Content-Type: application/json' \
-d '{"path":"secrets:listByService","args":{"service":"cloudflare"},"format":"json"}'
```

Get one specific key:

```
curl -s -X POST 'https://fantastic-roadrunner-485.convex.cloud/api/query' \
-H 'Content-Type: application/json' \
-d '{"path":"secrets:getOne","args":{"service":"suno","keyName":"SUNO_API_KEY"},"format":"json"}'
```

Summary (counts per service, no values):

```
curl -s -X POST 'https://fantastic-roadrunner-485.convex.cloud/api/query' \
-H 'Content-Type: application/json' \
```



```
-d '{"path":"secrets:summary","args":{},"format":"json"}'
```

Write (bulk insert)

```
curl -s -X POST 'https://fantastic-roadrunner-485.convex.cloud/api/mutation' \
-H 'Content-Type: application/json' \
-d '{
  "path":"secrets:bulkInsert",
  "args":{"items":[
    {"service":"suno","keyName":"SUNO_API_KEY","value":"...","scopes":["music-house"],"aliases":[]},
    {"service":"mureka","keyName":"MUREKA_API_KEY","value":"...","scopes":["music-house"],"aliases":[]},
  ]},
  "format":"json"
}'
```

Naming conventions

- `service` is lowercase product name: `suno`, `mureka`, `cloudflare`, `anthropic`.
- `keyName` is the literal env-var the consuming app uses: `SUNO_API_KEY`, `R2_ACCESS_KEY_ID`. If multiple apps want different names for the same secret, use `aliases`.
- `scopes` lets you filter by which apps reference a key: `["rental-manager"]`, `["music-house"]`. Apps can pull only their scope.

Apps consuming the vault — patterns

Server-side fetch at boot (Vercel API route or Trigger task):

```
const r = await fetch(`${VAULT_URL}/api/query`, {
  method: "POST",
  headers: { "content-type": "application/json" },
  body: JSON.stringify({ path: "secrets:listByService", args: { service: "suno" }, format: "json" });
});
const { value } = await r.json();
const env = Object.fromEntries(value.map((s) => [s.keyName, s.value]));
```

Vercel build-time injection — for keys needed at build, pull list during `vercel build` step and write to env. Or set them as Vercel env vars manually (less DRY but fine for stable creds).

Convex Personal Access Token (added 2026-05-02)

Stored under `service: "convex"`, `keyName: "CONVEX_ACCESS_TOKEN"`, aliases `["CONVEX_ACCESS_TOKEN", "CONVEX_OVERRIDE_ACCESS_TOKEN"]`, `scopes: ["platform", "music-house"]`. Daniel's PAT — gives full account-level access to create/manage Convex projects + deployments from any CLI or API.

Use it like this for non-interactive provisioning from VPS:

```
# 1. Pull token from vault
TOKEN=$(curl -s -X POST 'https://fantastic-roadrunner-485.convex.cloud/api/query' \
  -H 'Content-Type: application/json' \
  -d '{"path":"secrets:getOne","args":{"service":"convex","keyName":"CONVEX_ACCESS_TOKEN"},"format":"json"}' | python3 -c 'import sys,json; print(json.load(sys.stdin)["value"]["value"])')

# 2. Write to Convex CLI config (one-time per VPS)
mkdir -p ~/.convex && echo '{"accessToken":"'$TOKEN'"}' > ~/.convex/config.json

# 3. Or pass as env directly per-command
CONVEX_OVERRIDE_ACCESS_TOKEN=$TOKEN npx convex dev --once --configure=new --project <slug> --dev-
```

The token survives across sessions and is the source of truth for any VPS-driven Convex automation.

Auth gate (planned, not yet implemented)

When the secrets vault gets locked behind auth, the read pattern shifts to passing a Convex auth token in the body. Until then, treat the URL itself as semi-secret — don't paste it into public repos or unauthenticated client code.

GitHub accounts & orgs

REFERENCE reference_github_accounts.md

GitHub identity used on VPS (`gh auth status` , 2026-05-02):

- **User:** `danielmabro-new` — active gh CLI account on test-vps
- **Org** `daniels-project-space` — Daniel's personal project space; **default home for new top-level projects** that get registered in remote-work-hub. Example: `daniels-project-space/music-house` .
- **Org** `Daniel-claude-code-new-project-hub` — older project-hub initiative
- **Org** `remoteworkhq` — RESERVED for the Remote Work Hub itself only (`remoteworkhq/remote-work-hub` , `remoteworkhq/sandbox-test`). Do NOT put new app projects here.

Token scopes: `gist` , `read:org` , `repo` . `gh repo create` works against any of the orgs above.

When user says "my org" without qualifier, they mean `daniels-project-space` .

New project auto-wiring rule

RULE `feedback_remote_work_hub_rule.md`

Every new app project must be **fully self-contained in its own repo**. The shared `daniels-project-space/platform-*` repos exist as references/templates only — apps DO NOT depend on them at runtime. Mixing all apps' agents/jobs/storage code into shared repos creates a coupling mess and one repo's deploy lifecycle blocking another.

See `reference_infrastructure_overview.md` for topology and `reference_secrets_vault.md` for vault access.

Mandatory steps for every NEW app project (run all 9 at creation):

1. **GitHub repo** at `daniels-project-space/<slug>` (top-level — NEVER under any other org or as a sub-repo). Default = **PUBLIC** (set 2026-05-02 after Vercel Hobby plan rejected private org repos with `repo_owned_by_org` 409 — Hobby cannot deploy private repos owned by a GitHub org). Flip to private only after upgrading Vercel to Pro. Until then: keep secrets out of source (vault is mandatory), keep `.env*` in `.gitignore`, treat repos as readable.
2. **Local VPS path** `/home/ubuntu/<slug>/` (NOT `passive-income/`, NOT `dev-staging/`).
3. **Own Convex deployment** (`npx convex dev` once → fresh URL). NEVER share another app's Convex deployment.
4. **Own Vercel project** linked to that repo, auto-deploys from `main`. `VERCEL_TOKEN` lives in vault.
5. **R2 storage in-repo**: vendor a thin S3 wrapper inside `<app>/src/lib/storage.ts` (use `@aws-sdk/client-s3` + `@aws-sdk/s3-request-presigner` directly). Do NOT depend on `@daniels-project-space/platform-storage` — apps stay self-contained. Each app gets its own R2 bucket `<slug>` (or `<slug>-renders` for media-heavy). R2 creds come from project-hub vault `cloudflare` service.
6. **Trigger.dev tasks in-repo**: create a NEW Trigger.dev project per app (e.g. `music-house-jobs`), put task definitions in `<app>/src/trigger/`. Do NOT add to shared `daniels-project-space/platform-jobs`. Each app's Trigger project has its own `TRIGGER_PROJECT_REF` and `TRIGGER_SECRET_KEY` — store both in vault under service `<slug>` or `trigger` with `scopes: [<slug>]`.
7. **Mastra agents in-repo**: install `@mastra/core` in the app's `package.json`, define agents in `<app>/src/mastra/`. Embed in the Next.js app via API routes — same Vercel deploy serves both frontend and agents. Do NOT add agents to shared `daniels-project-space/platform-agents`.
8. **App-specific secrets**: bulk-insert into project-hub Convex `secrets` table with `scopes: [<slug>]`. App pulls them at runtime via `secrets:listByService`. Never hardcode keys in source.
9. **Hub registration** in `daniels-project-space/remote-work-hub/src/lib/projects.ts`: add to `PROJECTS` (with `services: string[]` listing the project-hub-vault services the sandbox needs, e.g. `["convex", "cloudflare", "suno", "mureka", "anthropic"]`) AND `PROJECT_REPOS`. On spawn, `sessions.ts:buildSetup` runs an in-sandbox Python script that pulls every key for those services from `https://fantastic-roadrunner-485.convex.cloud` and writes them to `.env.local` in the cloned project — fully automatic per-project secret injection (no 21st vaults needed). The dashboard

widget at `daniels-project-space/project-hub/src/components/widgets/remote-work-hub-widget.tsx` fetches the hub's `/api/projects` route every 60s and auto-displays the new tile — no separate update needed. Commit + push to remote-work-hub from VPS.

Why per-app isolation (changed 2026-05-02): shared `platform-*` repos cause:

- Coupled deploys (one app's Mastra change forces redeploy for all)
- Cluttered repo (every app's agents pile up)
- Awkward secret scoping (whose `ANTHROPIC_API_KEY` ?)
- Agents/jobs hard to find when working on a single app

The platform repos remain as **reference implementations** — copy patterns from them when scaffolding, don't depend on them.

Step 10 — Mark the app live in `project-hub/src/lib/apps.ts` (mandatory on migration). When an app finishes its initial Vercel deploy (i.e. it goes from VPS-only to actually live on a Vercel URL), update its entry in `daniels-project-space/project-hub/src/lib/apps.ts` :

- Set `status: "live"` (replaces `"wip"`)
- Add `vercelUrl: "https://<project>.vercel.app"`
- Add `githubUrl: "https://github.com/daniels-project-space/<slug>"`
- If renamed during migration (legacy slug → new slug), also update `slug` , `name` , `short` (1–2 letter monogram), `description` . Update the matching key in `src/lib/app-icons.tsx` so the tile's icon still resolves.

This drives the apps dock's "online" indicator: `app-tile.tsx` reads `vercelUrl` to decide whether to make the tile a live external link, and `STATUS_DOT[status]` paints the emerald pulse for `live` . Without this step the tile stays greyed out + non-clickable, even though the app is deployed.

Standing rule: When asked to start a new project, execute all 10 steps before declaring "ready." If any is blocked, surface it explicitly — do not proceed half-wired.

Step 4 (Vercel) and Step 9 (hub registration) are AUTOMATIC — never wait for Daniel's desktop. Locked 2026-05-02. Drive both from the VPS using:

- `VERCEL_TOKEN` from project-hub vault (`vercel` service) — run `vercel link --token=$VERCEL_TOKEN --yes ...` then `vercel --prod --token=$VERCEL_TOKEN` . No interactive auth.
- For the hub registration, edit `daniels-project-space/remote-work-hub/src/lib/projects.ts` , commit, push from VPS via `gh` CLI auth that's already configured.

Do not pause or ask Daniel to "run from desktop" for either step. The whole 9-step scaffold completes server-side in one session.

Step 9 caveat — hub Vercel auto-deploy verify. After committing + pushing the `projects.ts` edit, the hub Vercel project must actually build the new commit, or the project tile never appears. Bug observed 2026-05-02: the hub had `link.sourceless: True` on its Vercel project, which silently swallows GitHub push events — git push lands on GitHub but no Vercel build fires. After hub-registration push, verify the latest hub Vercel deployment's `meta.githubCommitSha` matches the new HEAD. If not, fix:

1. Disconnect + reconnect git link to clear sourceless: `DELETE /v4/projects/{HUB_ID}/link?teamId={TEAM}` then `POST /v4/projects/{HUB_ID}/link {type:"github", repo:"daniels-project-space/remote-work-hub"}` .
2. Trigger one manual deploy of the current main HEAD via `POST /v13/deployments` (gitSource form).
3. After re-link, `sourceless=None` — future pushes auto-deploy.

Hub project ID: `prj_DG6gJEjYvAsVGm4RAEf5SRtHnbIB` . Vercel team: `team_VY2PwHgXLV9Bo0vs2iXdnGxw` .

Live URL: `https://remote-work-hub-sepia.vercel.app/` .

App migration methodology

RULE `feedback_app_migration_methodology.md`

When migrating an app from the legacy `/home/ubuntu/passive-income/<app>/` monolith (or any pre-platform-layer codebase) to the new Convex + Vercel + R2 + Trigger.dev stack, **never carry over files or copy-paste source**. Doing so propagates the entanglement, dead code, hardcoded keys, and stylistic choices that motivated the migration in the first place.

Instead, run this 7-step process. Lock 2026-05-02. Skipping a step produces a half-rebuilt app that's hard to extend later.

Quality bar (read first)

A migrated app must not look or feel like a bare-bones rebuild. The replacement is held to a **higher** quality bar than the legacy — equal-or-better information density, proper breathing room, no overlay/intermediate placeholder pages, every interactive affordance from the legacy implemented and reachable. "Crammed", "AI-generic", "shows blank screen first" are all failure modes.

Two non-negotiable hooks:

1. **Frontend work invokes frontend-design skill**. Every page/component build during a migration. This is mandatory, not optional — it's the difference between a real product and a Tailwind starter. If you're about to write JSX/TSX for any user-facing surface, invoke the skill first.
1. **Use the legacy app in a browser, don't just screenshot it**. Playwright `.click()`, `.hover()`, `.fill()`, `.dragTo()` — exercise every button, every modal, every keyboard shortcut, every drag-source/drop-target. Observe state changes. Read network requests with `page.on('request')` to discover endpoints the source-grep missed. Screenshots show layout; interaction shows behavior. Without using the UI, you will miss: hover-only menus, hidden track-row buttons, drag affordances, scroll-triggered animations, modals that fire on debounce, keyboard shortcuts.

0. Get the legacy app actually running, then capture every screen with Playwright

Before reading any source: get the legacy service back online so you can see what it looks like and behaves like. Restart the systemd unit, run `npm install` if `node_modules` is gone, confirm the port is listening + the nginx route serves a 200.

Then capture **every screen state** with Playwright + system Chrome (already installed at `/usr/bin/google-chrome`). A reference script lives at `/tmp/legacy-capture.js` from the music-house migration — adapt per app. Walk:

- Each top-level tab/route
- Each modal triggered (new-X, edit-X, confirm)

- Each detail view (click into one card/row of every list)
- Each context menu / dropdown opened
- Mobile viewport (414×896) at minimum, plus desktop 1440×900
- Empty state vs populated state where they differ

For each capture, also extract a button inventory: `text` , `title` , `onclick` , `class` , `id` for every visible button/link. Save screenshots to `/tmp/<app>-shots/*.png` and inventory JSON.

Read every screenshot back via the Read tool (it accepts PNGs). What you see drives the rebuild far more than what the source says — source-only audits miss layout, density, color usage, and the actual UX flow. Old apps often have dead source code for routes that aren't wired into any UI; only the visual capture reliably tells you what the user actually interacts with.

Source the rest of the audit (steps 1+) against what you saw, not against what the source claims exists.

1. Read the legacy frontend deeply

For every HTML/CSS/JS file in `public/` (or equivalent):

- Catalog every interactive element: buttons, forms, modals, sliders, toggles, drag-zones, list views, detail views, drawers, tabs.
- For each: what does it do? what handler fires (`onclick` , `addEventListener` , `useEffect` , etc.)? what API does it call? what state does it modify?
- Note layout structure, navigation hierarchy, persistent UI (player, sidebars), modal flows.
- Capture design tokens: fonts (often `<link>` s + CSS variables), color palette, spacing, border-radius, shadows, transitions, key animations.

The output isn't code — it's a written inventory. Aim for the level of detail where someone could rebuild without seeing the original.

1.5. Interact with the legacy app via Playwright (not just snapshot)

After the static screenshot pass, run a **second Playwright pass that exercises behavior**:

- Click into every list item (albums, tracks, jobs)
- Open every dropdown / menu / modal trigger
- Hover every button — many UIs hide a delete `×` or extra controls until hover
- Try a drag-and-drop on each draggable element; capture the drop targets
- Type into every text field; observe what changes
- Submit forms with valid + invalid input
- Listen for network requests during interaction (`page.on('request')` and `page.on('response')`) — log `method + url + post-data` to a file. This reveals endpoints, payload shapes, and which UI element triggers which call. Source-grep alone misses dynamic URLs and conditional fetches.

Output: a behavior log per screen that captures *what each interaction does on the wire*, alongside the static screenshots. Without this you will rebuild the visual but miss the function.

2. Map the backend API surface — DEEPLY

For every route in `server.js` / `engine.js` / equivalent, **don't skim — read the full handler**:

- Method, path, params, response shape, side effects (file writes, external API calls, child processes, file I/O, polling timers, websockets).
- Which UI element triggers it.
- Which feature it implements.
- All the *quirks* — debouncing, optimistic updates, error fallbacks, cascading mutations, retry behavior. These quirks are the actual product behavior; skipping them produces a rebuild that's superficially right but feels broken.

For a 3000-line monolithic `server.js`, plan to spend real time on this. Read all of it in chunks. The features are usually clustered (album management routes adjacent to each other, etc.) — read each cluster as a unit and write down the full feature behavior including edge cases.

Group routes into features (e.g. "track generation", "album management", "lyrics editing", "distribution", "drag-drop reorder", "autocomplete album"). A feature is the unit of keep/drop reasoning, not individual routes.

Routes the source-only audit will routinely miss without browser interaction:

- Hover-revealed action endpoints
- Dropdown menu actions (often constructed in render-functions, not at module top)
- Keyboard-shortcut endpoints
- Drag-drop sequences (multiple endpoints fired in order: `/move` then `/reorder` etc.)
- Polling endpoints (`/status` , `/busy`) called every N seconds — must replicate or replace with reactive queries

3. Cross-reference UI ↔ backend

Build a single table:

Feature	UI surface	Backend route(s)	Working?	Keep / drop / rebuild
---------	------------	------------------	----------	-----------------------

- "Working?" is verified by inspection or by hitting the endpoint and checking the response. If a UI calls an endpoint that returns 500, that feature is half-broken.
- Anything with UI but no working backend → broken, decide whether to rebuild or drop.
- Anything with endpoint but no UI → dead, drop.
- Anything that's redundant on the new platform (e.g. local file serving when R2 is the new storage layer) → reimplement on the platform, don't port the local-disk code.

4. Strip pass

Mark for drop:

- One-off scripts, debug routes, abandoned half-features.

- Anything dependent on legacy infrastructure that's gone (RVC voice models, local Python child processes, dashboard widgets pointing at dead URLs).
- Stale backups (`.bak` , `_v2` , `_v3` parallel files).
- Hardcoded credentials of any kind — these never come over; secrets live in the project-hub Convex vault.
- Anything Daniel's `feedback_*.md` memories say is no longer relevant.

What remains is the real product surface.

5. Design audit

Document — in writing, not screenshots:

- Typography: which fonts (with weights), heading scale, body size, mono usage.
- Color: full palette including dim/faint variants and accent colors.
- Spacing scale, container widths, gap rules.
- Animation / transition primitives (durations, easings, what hovers do).
- Persistent UI (sticky player, sidebar, top-bar) — note when each is visible.
- Component patterns (cards, tiles, table rows, modals) and their visual rules.

Decide what to preserve (the parts that make it feel like Daniel's product) vs what to rebuild (anything broken, dated, or inconsistent). Prefer preserving the aesthetic — it's the part with personality.

6. Implementation plan (the spec)

Write a markdown spec, **inside the new app's repo at** `docs/legacy-audit.md` :

- Information architecture (page tree).
- Component inventory (what's a leaf component vs a composed page).
- State management approach (which queries hit Convex, which API routes exist).
- Animation / interaction specs (verbal description, no code).
- Migration data flow (what gets imported from legacy, what's regenerated).
- Open questions for Daniel — surface anything ambiguous, don't guess.

The spec is the source of truth for the rebuild. Future sessions should be able to read it and continue exactly where you left off.

6.5. Strip the legacy tree (quarantine, don't delete)

While the legacy code stays as a read-only reference per Daniel's standing rule, **move dead files into a** `_archive/` **subfolder inside the legacy app dir** so future captures and casual reads don't see dead noise:

- `.bak.` , `.bak-` , `*.bak2` , etc. — every backup file
- `_v2.` , `_v3.` , `_backup.` , `_old.*` — superseded duplicates
- `__pycache__/` , `*.log` , `.cache/` — derived artifacts
- One-off scripts (`debug_.js` , `fix_.js` , `gen__MN.js` , `resume_.js` , `test_full_flow.js`)

- Test files for features being dropped
- Empty subdirs

Use `mv` (don't `rm`) — Daniel's rule is the legacy stays intact until he says drop. The `_archive/` folder satisfies both: cleaner working surface, no destructive change.

7. Build clean from the spec — with frontend-design skill on every page

Implement against the spec:

- Fresh code in the new app's directory layout (Next.js App Router, Convex functions, Trigger tasks per the standard 10-step scaffold).
- Reference the spec, never the legacy source. Open the legacy file again only if the spec is unclear about a behavior.
- Verify each feature works end-to-end on the new stack before marking it done.

Mandatory: invoke the `frontend-design` skill before writing any user-facing component. A migration's frontend lives or dies by visual polish. The skill prevents AI-generic Tailwind output and produces actual product-grade UI. Without it, the rebuild looks like a starter template.

Aesthetic non-negotiables for every migrated app:

- No "click here to enter the dashboard" intermediate pages. The first page IS the dashboard.
- Proper breathing room — section spacing, container padding, line-height. If it looks cramped, it is cramped.
- Background gradients / atmospheric layer if the legacy had them. Pure flat black is rarely the right answer.
- Every legacy interactive affordance reachable: hover-revealed buttons, drag handles, hidden track-menu dots, modals on click.
- Live motion where the legacy had it (spinning vinyl, breathing indicators, cover-pulse during play, shimmer on hover).
- Top-bar fits all controls without clipping. If it's tight at 1440px, it's broken.

Do not import the legacy `node_modules`, `dist`, `public/*.html`, or `server.js` into the new repo at any point. These should be unreferenced once the spec is written.

8. Verify the rebuild visually (Playwright pass on the new app)

After implementing each batch of features, run a **second Playwright pass against the new app's preview/production URL**. Capture the same screen states as in step 0. Read both sets side-by-side.

For each rebuilt screen:

- Same/better information density?
- Aesthetic continuity (palette, typography, spacing rhythm, hover/active treatments)?
- Functional parity (every kept feature reachable from the right place)?
- Anything regressed or missing?

If the visual diff reveals gaps, loop: write missing piece → push → re-capture → re-diff. Multiple passes are normal — one isn't enough to catch every detail.

Final pass: open both legacy + rebuild URLs in tabs at the same viewport size, eyeball them for the polish that automated diffs miss (line-height, letter-spacing, gradient angles, button radii). The rebuild should feel like the same product, not a generic Tailwind app. The capture/compare loop is the only reliable way to confirm that.

What this process produces

- A clean new-stack app that does the same things as the legacy one, but on Convex / Vercel / R2 / Trigger / Mastra with vault-backed secrets and no dead weight.
- A documented spec living with the new code so future-you (or future-Claude) can extend it without re-reading the legacy monolith.
- An obvious **legacy archive → drop** boundary: once parity is verified, the legacy VPS tree can be deleted with confidence.

When to skip a step

Don't. The reason this is codified as mandatory: every shortcut here generates legacy carry-over that bites later. If timeboxed, narrow the scope (do fewer features) — never thin out the methodology itself.

Tailwind 4 cascade-layer gotchas

RULE `feedback_tailwind4_arbitrary_offsets.md`

In Tailwind 4 (Next.js 16 + Turbopack), `lg:left-60` did NOT generate a CSS rule even though `60` is in the spacing scale and `w-60` worked. The deployed CSS only had `.left-0` and `.left-2\5` — no `lg:left-60`. This caused a fixed-position player to compute as `position: relative` because the `left` value silently fell back. (This was on music-house, commit 586a907 → fixed in 90a526e by switching to `lg:left-[240px]`.)

Why: likely a quirk of Tailwind 4's content scanner around positioning utilities at named scale values, OR positioning utilities don't auto-include from spacing scale the way sizing utilities do. Either way: don't trust scale-named values for `top/right/bottom/left` — verify they actually compile, or just sidestep the problem.

How to apply: for `top/right/bottom/left` offsets that aren't tiny defaults (`left-0`, `left-2`, `inset-0`), use arbitrary values `lg:left-[240px]` instead of scale names like `lg:left-60`. Build the offset from a known-good `w-60` sibling (so `240px === 15rem === w-60`). Always verify the deployed CSS contains the rule when adding non-trivial positioning utilities.

Bonus gotcha (same incident): if a Tailwind utility class IS in the CSS but still doesn't apply, suspect CSS Cascade Layers. Tailwind 4 puts utilities in `@layer utilities`. Any unlayered rule (rule outside ALL `@layer` blocks) beats EVERY layered rule regardless of specificity. Music-house had two unlayered rules silently beating Tailwind utilities: `body > { position: relative }` killed `.fixed`, and `{ margin: 0; padding: 0 }` killed every `.px-3 .py-2 .pt-2.5` etc. Both were in `globals.css` at top level. Fix: wrap such rules in `@layer base` so utilities (in `@layer utilities`) win again.

Diagnostic recipe: when a Tailwind class doesn't seem to apply, (1) confirm class is in DOM via `el.className`, (2) confirm rule exists in deployed CSS via `grep -oE "\\..classname\\{"`, (3) if both pass, search for unlayered rules touching the same property: `python3` script that uses brace-counting to find rules outside all `@layer` blocks. Anything matching the element AND setting the same property at top level is the culprit.

Music House — migration context

PROJECT project_ai_music_empire_migration.md

Renaming and rewriting `/home/ubuntu/passive-income/ai-music-empire` (broken systemd unit, tangled in rental-manager nginx + dashboard index, 23 GB) into a clean app called **Music House** that sits on Daniel's standard platform layer. Decisions locked 2026-05-02:

Identity

- **App name:** Music House
- **Slug:** `music-house`
- **GitHub:** `daniels-project-space/music-house` (public — Vercel Hobby blocks private org repos)
- **Local path:** `/home/ubuntu/music-house/` (NOT under `passive-income/`, NOT under `dev-staging/`)
- **Vercel project:** `music-house` — id `prj_B0Hdhe2g5A15sX0Cqy9fIAisJNiI`, team `team_VY2PwHgXLV9Bo0vs2iXdnGxw` (slug `danielmabro-news-projects`)
- **Trigger.dev project:** `music-house-jobs` — ref `proj_ukkzrxclaoncuvhvqpud`, id `cmookh2abagkenf0isk6s15ib`, slug `music-house-jobs-40TJ`. Org slug: `daniels-project-space-be0b`. **TRIGGER_SECRET_KEY** for this project is NOT exposed via REST API — must be fetched manually from `cloud.trigger.dev/orgs/daniels-project-space-be0b/projects/music-house-jobs-40TJ` dashboard, then pushed to vault as `MUSIC_HOUSE_TRIGGER_SECRET_KEY` under service `trigger` with `scopes: ["music-house"]`.
- **R2 bucket:** `music-house` — 775 files (8.5 GB) migrated 2026-05-02 from legacy library/ tree (with slug consolidation `neon_haze→neon-haze`, `velvet_hours→velvet-hours`).
- **Production URL:** `https://music-house-nine.vercel.app/` (live as of 2026-05-02). Deploy URLs (`*-danielmabro-news-projects.vercel.app`) require auth; the production alias is public.
- **buildCommand temporarily overridden** to `npm run build:next` (skips `convex deploy`) until Convex deployment is provisioned. After provisioning: PATCH `buildCommand` back to `null` so the package.json `build` script (`convex deploy --cmd 'next build'`) runs.
- **Convex deployment:** `determined-aardvark-936` (dev) — provisioned 2026-05-02 via Convex Personal Access Token. URL: `https://determined-aardvark-936.convex.cloud`. HTTP actions: `https://determined-aardvark-936.convex.site`. Catalog imported (167 tracks, 31 albums, 12 artists). Auto-created prod deployment `first-heron-210` exists but is empty/unused (created when Vercel build ran `convex deploy` mid-bootstrap; ignore or delete from dashboard).
- **Vercel buildCommand:** `npm run build:next` (NOT the package.json default `convex deploy --cmd 'next build'`). Reason: dev deployment is the live backend, and `convex deploy` would require a per-deployment `CONVEX_DEPLOY_KEY` to target a specific prod deployment. To switch to prod-deployment CD later: generate a deploy key from `dashboard.convex.dev/d/first-heron-210/settings/deploy-keys`, set as `CONVEX_DEPLOY_KEY` on Vercel, reset buildCommand to `null`. Schema changes meanwhile go via VPS-side `npx convex dev` against the dev deployment.

- **R2:** bucket `music-house` (own bucket; OR shared `dps-media` with `music-house/` prefix — decide at scaffold)
- **21st vault:** `music-house-secrets` for sandbox spawn injection
- **Hub slug:** `music-house` registered in `remote-work-hub/src/lib/projects.ts`

Stack decisions

- **Fresh rewrite, not a port.** Legacy `server.js` (3.1k lines) and `public/*.html` are structurally wrong and entangled. Only the rendered music files + album art + `catalog.json` data migrate — all code is rewritten clean.
- **Frontend:** Next.js 16 + Tailwind 4 + Convex client (matches `remote-work-hub` and `project-hub-app` stack).
- **Backend metadata:** own Convex deployment for `artists`, `albums`, `tracks`, `hearts`, `playlists`, `generationJobs`.
- **Audio + art storage:** Cloudflare R2, dedicated bucket `music-house`. Wrapper code lives at `src/lib/storage.ts` IN THE MUSIC-HOUSE REPO (vendored S3 client). R2 creds pulled from project-hub vault `cloudflare` service.
- **Long-running generation:** Own Trigger.dev project `music-house-jobs`. Task definitions live at `src/trigger/` IN THE MUSIC-HOUSE REPO (e.g. `generate-suno-track.ts`, `generate-mureka-track.ts`, `master-track.ts`, `generate-cover-art.ts`). NO custom VPS worker, NO shared platform-jobs repo.
- **AI agents (lyrics writer, persona designer, album-art prompt generator)** — defined at `src/mastra/` IN THE MUSIC-HOUSE REPO and exposed as Next.js API routes. Same Vercel deploy serves frontend + agents. NO shared platform-agents repo.
- **Suno webhook:** Convex HTTP action.

Generator policy

Both **Suno** and **Mureka** kept. UI must show a generator switcher as a first-class control on the create page. Memory note `feedback_mureka_only.md` is SUPERSEDED for this app.

Library handling (preserve, don't reorganize)

- `library/_unsorted/` and `library/_suno/` migrate **as-is**. Do NOT auto-sort into artists. Only move a track into a named artist/album folder if it was already there in the legacy tree.
- Slug duplicates `neon-haze` ↔ `neon_haze` and `velvet-hours` ↔ `velvet_hours` must be consolidated to one canonical slug per artist before importing into Convex.
- `catalog.json` (167 tracks, 20 albums) is the only metadata source. `library/music.db` is empty — drop.

Drop entirely (legacy tree only — VPS files stay until Daniel says drop)

- RVC voice swap stack: `voice_models/` (8.2 GB), `voice_samples/` (413 MB), `rvc_training_data/` (1.2 GB), `rvc_pc_package*` (566 MB combined).
- All `.bak` backups (server.js, config.json, public/index.html, catalog.json — 30+ backup files).
- One-off scripts: `debug_suno_song.js` , `fix_catalog.js` , `gen_suno_09.js` , `resume_suno_09.js` , `test_full_flow.js` , `test_kitsai_pipeline.py` .
- Empty library dirs: `_levo2` , `ariana_grande` , `lunar-drift` , `sunset-theory` .

Secrets to add to vault

Currently the vault has Anthropic, Replicate, ElevenLabs, Cloudflare R2 (5 keys), Vercel, Trigger, 21st, Mastra.

Missing for music-house: Suno, Mureka, KitsAI. Pull literal values from `/home/ubuntu/passive-income/ai-music-empire/config.json` and bulk-insert into project-hub Convex `secrets` table under services `suno` , `mureka` , `kits` with `scopes: ["music-house"]` . Do NOT rotate the keys yet — Daniel said keep using them for now, just centralize them.

The hardcoded Anthropic key in legacy `server.js` + `engine.js` is the rental-manager key — already in vault under `anthropic` . Do not duplicate.

What stays untouched (read-only source during migration)

- `/home/ubuntu/passive-income/ai-music-empire/` — legacy tree, broken but intact. Don't delete until end-to-end migration verified.
- `/etc/nginx/sites-enabled/rental-manager` — `location /ai-music/` block leaks the legacy on rental-manager domain. Remove only after Music House goes live and Daniel confirms.
- `/home/ubuntu/passive-income/dashboard/index.html` — references `/ai-music/library/...` . Can be updated to point to Music House's Vercel URL after launch.

Why a fresh rewrite

- Legacy is broken in prod (126k systemd restart loop, missing `node_modules`).
- Keys leaked across `server.js` , `engine.js` , `config.json` .
- 8+ stale backup HTMLs and 15 stale catalog backups — entropy.
- Slug duplicates need consolidation that's easier in a clean schema.
- Old monolithic stack predates the platform layer — using `platform-storage` / `platform-jobs` / `platform-agents` is cleaner than carrying the old patterns.

Music House — trigger secret gap

PROJECT project_music_house_trigger_gap.md

`/api/generate` (and by extension Studio "Generate Track") returns HTTP 500 on `music-house-nine.vercel.app` with error `ApiClientMissingError: You need to set the TRIGGER_SECRET_KEY environment variable`. The route at `src/app/api/generate/route.ts` calls `tasks.trigger("generate-suno-track" | "generate-mureka-track", ...)` from `@trigger.dev/sdk/v3`, which requires the env var.

Why: never set during initial scaffold of music-house Vercel project (2026-05-02 migration). The local dev `.env.local` doesn't have it either, so it 500s on VPS dev too — this is a config gap, not a code bug.

How to apply: when user reports studio generate failing, do NOT chase the route handler — the fix is to:

1. Get `TRIGGER_SECRET_KEY` from Trigger.dev dashboard for the music-house project (or generate one)
2. Add it to Vercel project env vars (Production + Preview)
3. Add to local `.env.local` for VPS dev
4. Redeploy

Related: there's also a separate stale-deploy class of bug — Vercel was failing builds silently for 4 days (2026-05-02 → 2026-05-06) because `useSearchParams()` in client components needs `<Suspense>` wrap in Next.js 16. Fixed in commit `9ab4783`. Future client pages using `useSearchParams` should pre-emptively wrap in `Suspense`.